AIMERZ.ai
Aim.Act.Achieve

NOTES

Fundamentals of Java Programming -4

(Methods Overloading & Compile-time Polymorphism)

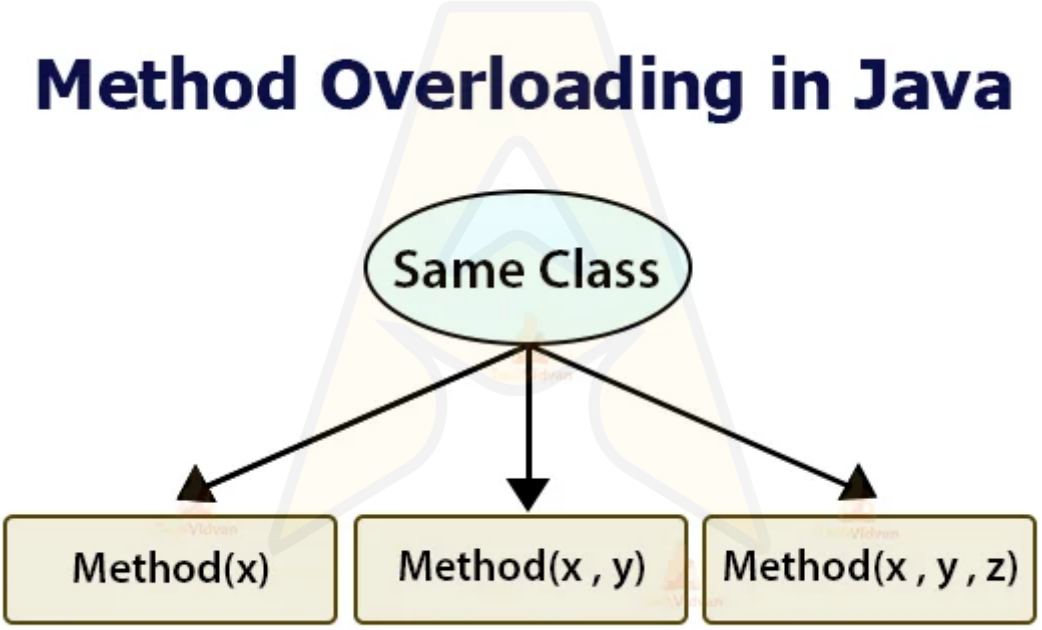
Method Overloading in Java

In Java, Method Overloading means we can have multiple methods with the same name in one class. However, each method should have different parameters. Parameters can vary by:

- The number of parameters (e.g., 2 or 3 numbers)
- The type of parameters (e.g., `int` or `double`)

Method Overloading is also called:

- **Compile-time Polymorphism**
- **Static Polymorphism**
- **Early Binding**



Why Use Method Overloading?

When we use method overloading, Java can decide which method to call based on the parameters we provide. For example, if we send two integers, it will call the method that works with two integers.

Example of Method Overloading in Java

In this example, we have three `sum` methods, each with different parameters:

```java
public class Sum {
    // This sum method takes two int parameters
    public int sum(int x, int y) {
        return (x + y);
    }

    // This sum method takes three int parameters
    public int sum(int x, int y, int z) {
        return (x + y + z);
    }

    // This sum method takes two double parameters
    public double sum(double x, double y) {
        return (x + y);
    }

    // Driver code to test the sum methods
    public static void main(String args[]) {
        Sum s = new Sum();
        System.out.println(s.sum(10, 20));         // Calls the method with two int parameters
        System.out.println(s.sum(10, 20, 30));     // Calls the method with three int parameters
        System.out.println(s.sum(10.5, 20.5));     // Calls the method with two double parameters
    }
}
```

Explanation:

1. sum(int x, int y) is called when two integer numbers are given.
2. sum(int x, int y, int z) is called when three integer numbers are given.
3. sum(double x, double y) is called when two decimal numbers are given.

Different Ways of Method Overloading in Java

In Java, we can overload methods in several ways. This means we can use the same method name but with different parameters to make our code more flexible and organized. Here are the main ways to achieve method overloading:

1. Changing the Number of Parameters

- We can create methods with the same name but a different number of parameters.
- For example, one `multiply` method might take two parameters, while another takes three.

```Java
public class MultiplyExample {
    // Method with two parameters
    public int multiply(int a, int b) {
        return a * b;
    }

    // Method with three parameters
    public int multiply(int a, int b, int c) {
        return a * b * c;
    }

    public static void main(String[] args) {
        MultiplyExample example = new MultiplyExample();
        System.out.println(example.multiply(2, 3));       // Calls method
with two parameters
        System.out.println(example.multiply(2, 3, 4));    // Calls method
with three parameters
    }
}
```

Output:
6
24

2. Changing the Data Types of the Parameters

- We can create methods with the same name but different data types for the parameters.
- For example, one multiply method might take int parameters, and another might take double parameters.

```java
public class MultiplyExample {
    // Method with int parameters
    public int multiply(int a, int b) {
        return a * b;
    }

    // Method with double parameters
    public double multiply(double a, double b) {
        return a * b;
    }

    public static void main(String[] args) {
        MultiplyExample example = new MultiplyExample();
        System.out.println(example.multiply(2, 3));        // Calls method
with int parameters
        System.out.println(example.multiply(2.5, 3.5));    // Calls method
with double parameters
    }
}
```

Output:
6
8.75

3. Changing the Order of the Parameters

- We can create methods with the same name and same data types, but in a different order.
- For example, print(String, int) and print(int, String) are treated as different methods because the order of parameters is different.

```java
public class PrintExample {
    // Method with String, int order
    public void print(String text, int number) {
        System.out.println("String: " + text + ", Number: " + number);
    }

    // Method with int, String order
    public void print(int number, String text) {
        System.out.println("Number: " + number + ", String: " + text);
    }
```

```
    public static void main(String[] args) {
        PrintExample example = new PrintExample();
        example.print("Hello", 5);          // Calls method with String, int
order
        example.print(10, "World");         // Calls method with int, String
order
    }
}
```

Output:
String: Hello, Number: 5
Number: 10, String: World

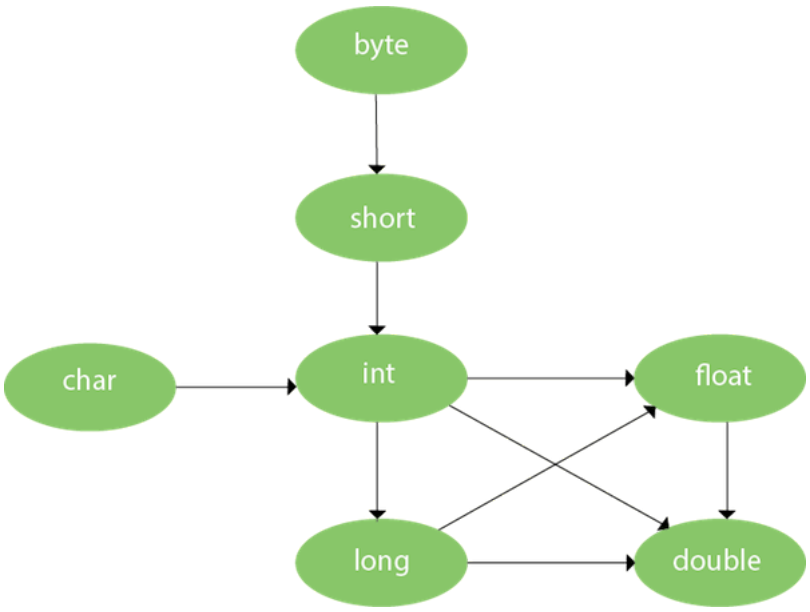
Type Promotion in Java

Type Promotion in Java is a process where Java automatically converts a smaller data type into a larger one when performing operations to prevent data loss. This typically happens in mathematical expressions or when passing arguments to methods.

How Type Promotion Works

Java follows a certain order for promoting data types:

- byte → short → int → long → float → double



For example:

- If we perform an operation with an int and a double, Java promotes the int to a double before performing the operation.

Examples of Type Promotion

1. **In Expressions**

```java
int a = 10;
double b = 5.5;
double result = a + b; // 'a' is promoted to double
System.out.println(result); // Output: 15.5
```

2. **In Method Calls**

```java
void display(double x) {
    System.out.println(x);
}

public static void main(String[] args) {
    int num = 7;
    display(num); // 'num' is promoted to double
}
```

Advantages of Method Overloading

- **Improves Readability:** Same method name with different parameters makes code easy to read and understand.
- **Reusability:** We can reuse method names for similar tasks, avoiding duplication.
- **Flexibility:** Overloading allows methods to work with different types or numbers of parameters.
- **Supports Polymorphism:** Enables methods to respond differently based on input, making code modular.

- **Better API Design:** Users can pick the method version that fits their needs, making APIs more user-friendly.

Disadvantages of Method Overloading

- **Ambiguity:** Similar parameter types can confuse the compiler.
- **Complexity:** Too many overloaded methods can make code harder to understand.
- **Unexpected Behavior:** Overloaded methods may cause confusion if not well-documented.
- **Slight Performance Impact:** Multiple overloads can slightly slow down runtime performance.
- **Overuse Can Lead to Code Bloat:** Excessive overloading can make code messy and hard to maintain.

Frequently Asked Questions about Method Overloading

Can we overload static methods?
Yes, we can overload static methods by changing their parameters (number, type, or order of parameters). However, we can't override static methods.

Can we overload methods that differ only by the static keyword?
No, methods cannot be overloaded by using the static keyword alone. The parameters must also be different.

Can we overload methods based only on the return type?
No, we cannot overload methods just by changing the return type. The parameter list must be different to create a valid overload.

Can we overload main() in Java?
Yes, we can overload the main() method by changing its parameters. However, the JVM will only call the standard main(String[] args) method to start the program. Other main() overloads can be called like regular methods within the program.

Compile time Polymorphism in Java

Polymorphism is one of the key concepts in Object-Oriented Programming (OOP), and it comes from the Greek words "poly" (many) and

"morph" (form). So, polymorphism means "many forms." In Java, polymorphism allows one entity (like a method or an object) to have multiple meanings or behaviour based on the context.

Why is Polymorphism Important?

Polymorphism provides flexibility and simplicity to code. It allows us to use a single interface (like a method name) to represent different forms or behaviour, making the code easier to read, manage, and extend.

There are two main types of polymorphism in Java:

1. **Compile-time Polymorphism (Method Overloading)**
2. **Run-time Polymorphism (Method Overriding)**

What is Compile-Time Polymorphism?

Compile-Time Polymorphism is also called **static polymorphism** or **early binding**. This type of polymorphism is determined during the compilation of the code, meaning it is fixed before the program runs.

In Java, compile-time polymorphism is achieved through:

- **Method Overloading** – Multiple methods with the same name but different parameters.
- **Operator Overloading** – Using operators in different ways (though Java has limited operator overloading).

Method Overloading

(which we already discussed earlier) allows multiple methods with the same name but different parameters.

Operator Overloading in Java

Note: Java does NOT support operator overloading. This means we cannot change how operators (like +, -, *, etc.) behave with user-defined objects. However, we can achieve similar functionality by creating methods that handle different types of input.

What is Operator Overloading?

Operator overloading is when an operator (like +) can perform different tasks based on the context. In languages that support it, we can change the meaning of an operator to work with custom objects or in new ways. Java does not allow us to redefine operators, but we can create methods that perform similar actions.

Example of Simulating Operator Overloading in Java

In this example, we'll use an add method to handle different inputs:

- Adding two numbers
- Concatenating (joining) two strings

```java
public class AIMERZ {

    // Method for adding two integers
    void add(int a, int b) {
        int sum = a + b;
        System.out.println("Addition of two integers: " + sum);
    }

    // Method for concatenating two strings
    void add(String s1, String s2) {
```

```
        String result = s1 + s2;
        System.out.println("Concatenated strings: " + result);
    }

    public static void main(String args[]) {
        AIMERZ example = new AIMERZ();

        // Adding two numbers
        example.add(10, 10);

        // Concatenating two strings
        example.add("Operator", " overloading");
    }
}
```

Output:
Addition of two integers: 20
Concatenated strings: Operator overloading

Explanation

In the above example:

- The add(int a, int b) method adds two integer values.
- The add(String s1, String s2) method joins two strings together.

THANK YOU

AIMERZ.ai

Aim.Act.Achieve